

Внутренняя Support-CRM

Архитектура и функционал проекта

MVP · white-label

1. Функционал на выходе (MVP)

Что система умеет уже в первой рабочей версии — по духу это Zendesk: обращения из разных каналов в едином инбоксе, обработка тикетов, автоматизации и отчётность.

Каналы (в составе MVP)

- **Live-chat виджет** — живой чат на сайте (встраиваемый).
- **Мобильный чат** — чат внутри мобильных приложений (iOS / Android SDK).
- **Email** — почта поддержки — входящие и исходящие.
- **API-канал** — приём диалогов от нашего внешнего AI-виджета через API.
- Мессенджеры (WhatsApp, Telegram, соцсети) — **на этапе расширения после MVP** (архитектура их поддерживает).

Работа с обращениями

- **Единый инбокс и тикеты** — статусы (открыт / в ожидании / решён / отложен), приоритеты, представления (views).
- **Инструменты агента** — шаблоны ответов, макросы, приватные заметки, @упоминания, метки.
- **Маршрутизация и авто-назначение** — round-robin по командам и нагрузке; переназначение при переоткрытии.
- **Автоматизации** — движок правил «если → то» (ключевые слова, теги, события).
- **SLA** — контроль времени первого ответа.
- **Бот и эскалация** — бот обрабатывает первичное, при необходимости эскалирует на агента.

Данные, доступ, аналитика

- **Контакты и компании** — с настраиваемыми полями (custom attributes).
- **Роли и права (RBAC)** — гранулярный доступ по областям; супер-админ управляет пользователями и правами.
- **Аудит-лог** — запись чувствительных действий (выгрузки, смена прав, удаления, доступ к данным).
- **Безопасные файлы и выгрузки** — проверка загрузок; выгрузки с ограничением по времени и журналированием.
- **Аналитика** — брендированные дашборды: объём, время ответа/решения, нагрузка, разрезы по каналу/метке/команде.

Категории обращений (таксономия тем)

Система рассчитана на многоуровневую классификацию обращений (уровни L1 → L2 → L3). Основные категории и типичные под-темы:

- **Депозиты** — статус депозита, «средства не зачислены», задержки, отклонённый платёж.
- **Вывод средств** — статус вывода, «не могу вывести», в ожидании, отклонён.
- **Верификация / KYC** — документы, проверка телефона и email, статус верификации.
- **Промо и бонусы** — приветственный бонус, кэшбэк, фриспины, депозитный %, отозванный бонус.
- **Аккаунт** — смена персональных данных, сброс пароля, ограничения, закрытие / самоисключение.
- **VIP** — перки, follow-up, персональные менеджеры.
- **Продукт и технические** — вход в аккаунт, сайт, спортбук, казино, технические сбои.

Поля обращения (настраиваемые атрибуты)

- **Платёжные** — сумма депозита, дата платежа, платёжный шлюз, PSP, ID транзакции, внешний PSP ID.
- **Идентификация** — Player ID, страна, язык, уровень пользователя.
- **Классификация** — тип обращения, тип проблемы, тема (topic).
- **Решение** — уровень и тип резолюции, требуемые задачи, статус согласования.

Метки, макросы, автоматизации

- **Метки (labels)** — семейства по категориям (депозиты, выводы, верификация, промо, VIP) и служебные (бот, авто-маршрутизация).
- **Макросы** — приветствия и закрытия, смена данных с/без KYC, сценарии account-safety (самоисключение), по стране/валюте, follow-up.
- **Автоматизации / триггеры** — маршрутизация по ключевым словам, назначение в группу, запрос оценки (CSAT), авто-переназначение, теги по событию.

Команды, роли, маршрутизация

- **Команды (группы)** — общая поддержка, депозиты, VIP, VIP-менеджеры (эскалация L1), команды по платёжным провайдерам.
- **Роли** — агент, администратор, супер-админ.
- **Маршрутизация** — омниканальная, round-robin, с учётом нагрузки; переназначение переоткрытых.
- **Представления (views)** — по статусу, VIP, депозиты, «на ревью супервайзера», бот-управляемые, эскалированные.
- **SLA** — время первого ответа.

- **Каналы** — веб-виджеты, мобильный чат (iOS/Android), email, API-канал; бот с эскалацией на агента.

Ключевые фишки (обзор уровня модулей)

Полный перечень возможностей MVP на уровне модулей (не отдельные функции кода):

- **Живой чат (live-chat)** — виджет на сайте и чат в мобильных приложениях.
- **Единый инбокс** — все каналы и очереди в одном окне агента.
- **Управление тикетами** — жизненный цикл: статусы, приоритеты, назначение, эскалация.
- **База контактов и компаний** — карточки клиентов с настраиваемыми полями.
- **Мульти-аккаунтность / мульти-бренд** — несколько брендов на одной платформе (white-label), у каждого своя тема.
- **Админ-панель** — управление пользователями, ролями, командами, ящиками, каналами, макросами, автоматизациями, SLA и брендингом.
- **Роли и права (RBAC)** — гранулярный доступ по областям, супер-админ.
- **Автоматизации и SLA** — правила «если → то» и контроль времени ответа.
- **Макросы и шаблоны ответов** — ускорение типовых действий агента.
- **Отчёты и дашборды** — брендированная аналитика.
- **Уведомления** — в приложении и по email.
- **Глобальный поиск** — по тикетам, контактам и сообщениям.
- **Публичный API и вебхуки** — интеграции с внешними системами.
- **Мобильные приложения** — чат через SDK (iOS / Android).
- **Мультиязычный интерфейс** — EN / ES.
- **Светлая и тёмная темы** — переключаемые.
- **Безопасность и аудит** — 2FA, аудит-лог, разграничение доступа.
- **Резервное копирование и хранение** — бэкапы и политика хранения данных.
- **Бот и эскалация** — авто-обработка первичного с передачей агенту.
- Опционально / после MVP: база знаний (help-center), CSAT-опросы, прямые коннекторы мессенджеров.

2. Архитектура системы

Модульный монолит на NestJS: единое приложение с модулями по доменам (тикеты, контакты, каналы, автоматизации, RBAC и т.д.). Отдельные сервисы (например AI) выделяются позже — по мере необходимости. Всё self-hosted, данные остаются в нашем контуре.

Поток данных: каналы → Caddy (единая точка входа + HTTPS) → фронтенд (Vue) и API (NestJS). API работает с базой через Prisma (S



upabase Postgres), с фоновыми задачами через Redis + BullMQ, с логинами и файлами — через Supabase Auth/Storage. Живые обновления идут по WebSocket/SSE.

3. Технологический стек

Инструменты и назначение каждого:

Слой	Технология	Что делает
Язык	TypeScript	Один типизированный язык на бэк и фронт — меньше ошибок.
Бэкенд	NestJS	Каркас серверной логики: тикеты, правила, API.
БД + Auth + Файлы	Supabase (self-hosted)	Данные (PostgreSQL), логины и файлы — на наших серверах.
Доступ к данным	Prisma	Безопасный типизированный доступ к базе.
Задачи	Redis + BullMQ	Фон: уведомления, выгрузки, регламентные задачи.
Реальное время	WebSocket + SSE	Живые обновления и стриминг AI-ответов.
Фронтенд	Vue 3 + Vite + Tailwind	Интерфейс агента и быстрая сборка.

UI-компоненты	shadcn-vue	Готовые блоки, код которых мы держим у себя.
Веб-сервер / HTTPS	Caddy	Маршрутизация и автоматический HTTPS.
Упаковка	Docker Compose	Одинаковый запуск сервисов на любой машине.
Деплой	GitHub Actions	Тесты и выкатка на сервер автоматически.

4. UI / UX

Интерфейс строится на shadcn-vue — бесплатной (MIT) библиотеке, код компонентов которой копируется в наш проект: мы им владеем и свободно стилизуем, без привязки к вендору и без платы. Единый вид держится на слое дизайн-токенов (CSS-переменные); фирменные цвета задаются один раз; есть светлая и тёмная темы.

Чего нет в библиотеке (крупные таблицы, графики, богатый ввод) — строим на «headless»-библиотеках и одеваем в те же токены. Идеи для экранов и эффектов берём из референсов (напр. Refero), перенося в наш стиль. Перед разработкой интерфейса — «дизайн-развилка»: несколько интерпретаций ключевого экрана, руководство выбирает направление, из него собирается брендбук.

Примеры компонентов shadcn-vue

- [Accordion](#) — сворачиваемые секции. ([оригинал shadcn/ui](#))
- [Dialog](#) — модальные окна.
- [Dropdown Menu](#) — меню действий.
- [Tabs](#) — переключение вкладок.
- [Command](#) — командная палитра / поиск.
- [Badge](#) — статус-пилюли и теги.
- [Card](#) — контейнеры контента.

Reactions

Use `BubbleReactions` for bubble reactions. You can use it to display reactions or quick action buttons. Use `side` and `align` to position the row — `side="top"` anchors it to the upper edge. Reactions overlap the bubble edge, so leave vertical space between rows — the examples below use a larger `gap` for this reason.

I don't need tests, I know my code works.
 

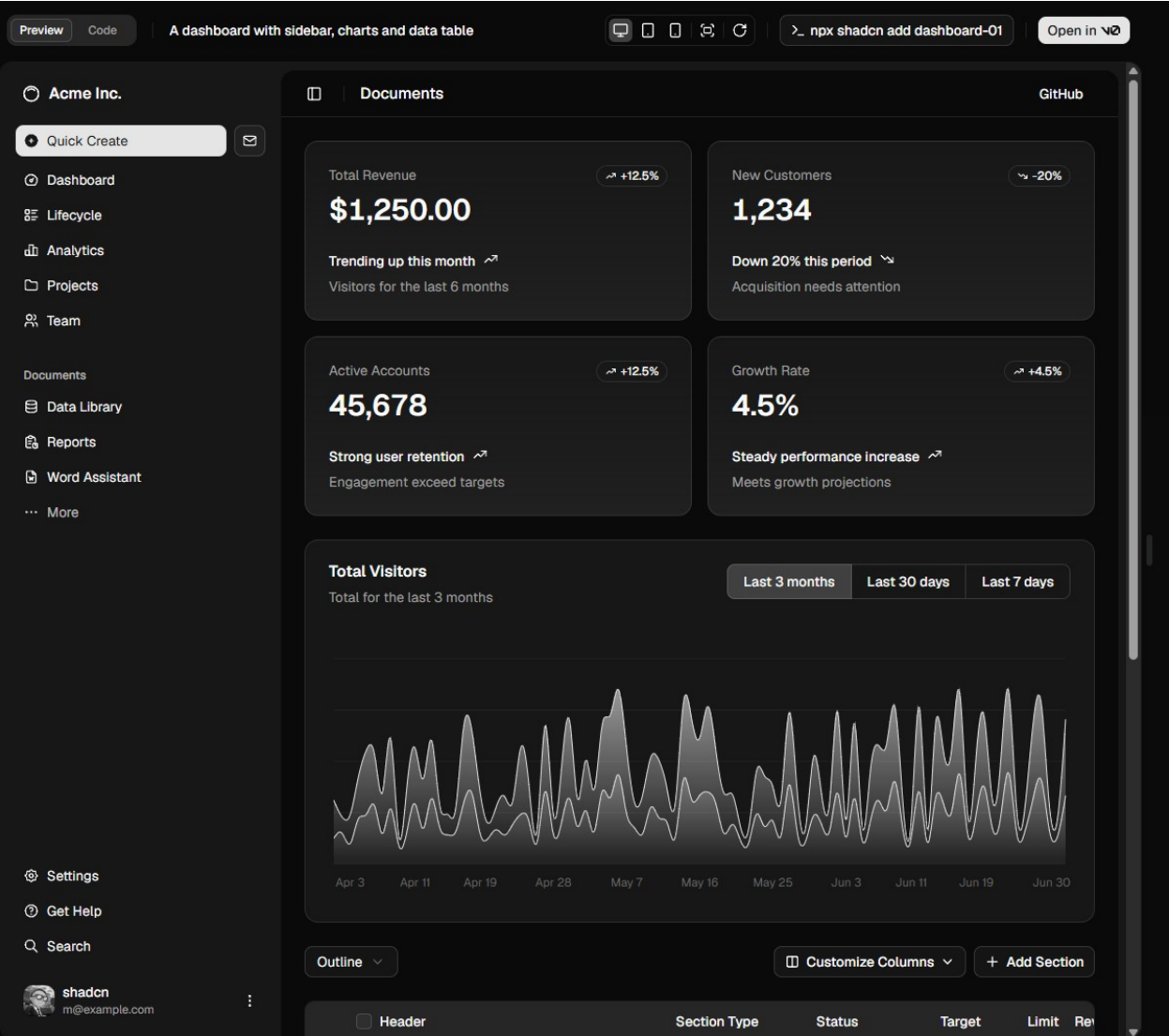
Bold. Fine I'll add some tests. I'll let you know when they're done.
  +2

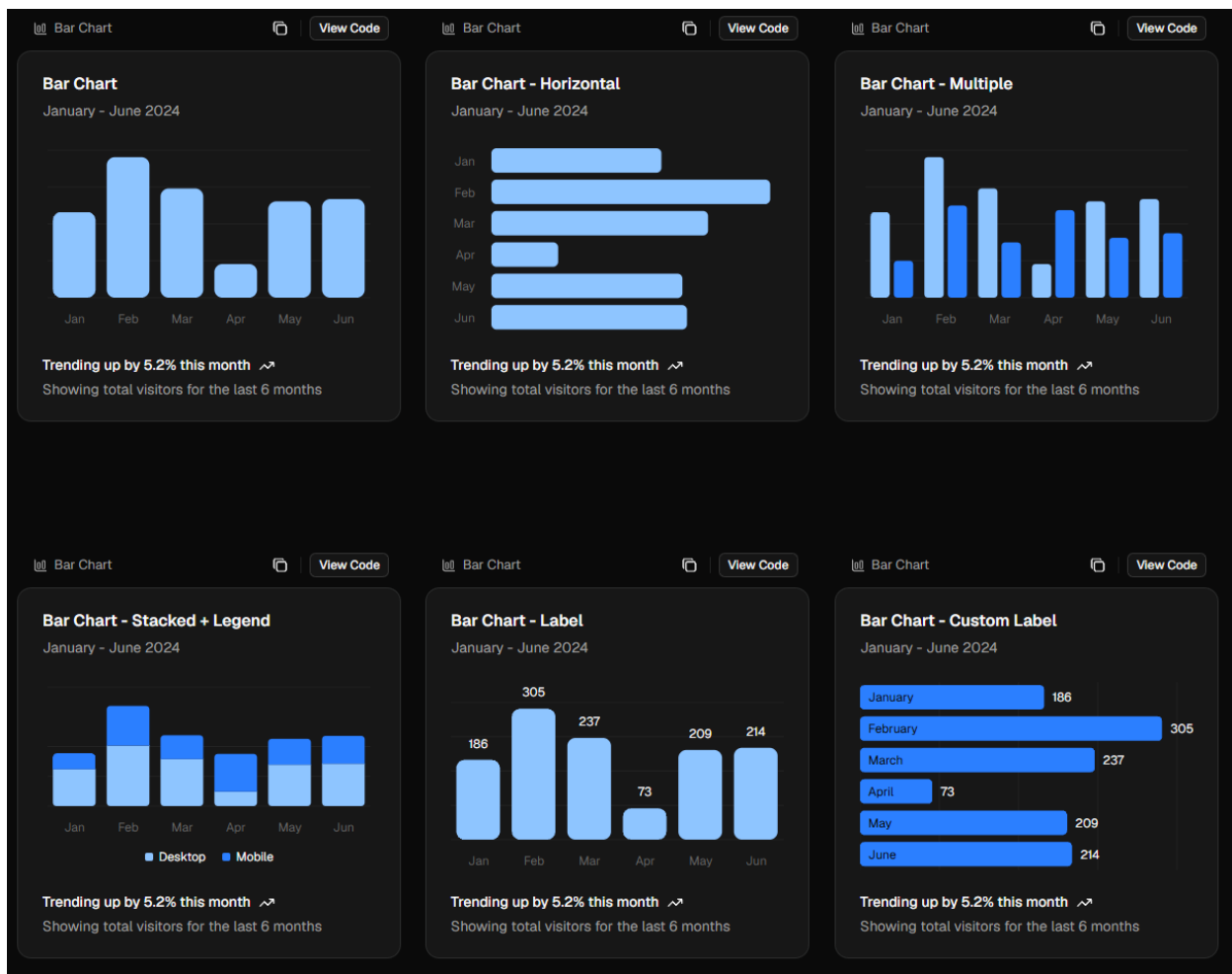
 
Tests passed on the first try. All 142 of them.
Looking good!

Are you sure I can run this command?
Yes, run it

```
1 <script setup lang="ts">
2 import { toast } from 'vue-
3 import { Bubble, BubbleContent, BubbleReactions } from '@components/ui/but
```

View Code





5. Что уже сделано — планирование и сбор данных

Перед разработкой пройден большой этап проектирования и исследования:

- **Архитектурные решения** — зафиксировано 28 решений (ADR): язык и фреймворк бэкенда, платформа БД/авторизации, фронтенд и подход к компонентам, деплой, мульти-бренд и др.
- **Security-разбор** — внешний аудит переоценён под нас: находки разложены на блокиеры (до реальных данных), важные фиксы до запуска, инфра-гигиену и комплаенс; заданы инварианты «сделать правильно».
- **Продуктовый scope** — определён функционал MVP и что выносится на потом.
- **Конфигурация из Zendesk** — разобрана текущая конфигурация (категории, поля, метки, макросы, автоматизации, роли, маршрутизация, SLA, каналы) — для повторения и миграции через API.
- **UI-подход** — выбраны shadcn-vue + дизайн-токены; запланирована дизайн-развилка.

- **База знаний** — документация и дорожная карта собраны в едином репозитории с корневым файлом-индексом.

6. Дорожная карта

Разработка идёт по фазам. Бэкенд и инфраструктура не зависят от дизайна и могут вестись параллельно с дизайн-развилкой; интерфейс — после выбора дизайна.

- **А. Дизайн-направление** — варианты ключевого экрана на реальных компонентах (Vue 3 + shadcn-vue); тема через CSS-переменные; выбор → брендбук (дизайн-токены: CSS custom properties + конфиг Tailwind).
- **В. Основания** — монорепо (api + web). Бэкенд на NestJS (модульная структура, DI, переписываем Ruby). Фронт — Vite + Vue 3 + Tailwind + shadcn-vue. Docker Compose поднимает self-hosted Supabase (Postgres, Auth/GoTrue, Storage) и Redis; Caddy — reverse-proxy с автоматическим TLS (Let's Encrypt). Секреты в .env (в .gitignore) и переменных окружения, без дефолтных паролей. CI на GitHub Actions (линт + тесты).
- **С. Данные и доступ** — схема БД в Prisma с версионными миграциями; account-scoring (задел мультиарендности) через middleware. Авторизация — Supabase Auth, JWT валидируется в NestJS-guard; поверх — свой RBAC (guards + политики по областям, роли: агент / администратор / супер-админ). Сессия в httpOnly-cookie, базовый CSP.
- **Д. Ядро поддержки** — модули NestJS: conversations, contacts, inboxes, teams, macros, automations, sla, audit. Доступ к данным через Prisma; фоновые операции — BullMQ поверх Redis. Файлы — Supabase Storage с проверкой типа/размера; выгрузки — по ссылкам с истечением; аудит-лог чувствительных действий.
- **Е. Каналы** — API-канал: REST-эндпоинты + входящие вебхуки с проверкой подписи. Веб-виджет: встраиваемый JS-скрипт + WebSocket. Мобильный чат: SDK (iOS/Android) поверх того же API. Email: приём/отправка (IMAP/SMTP или провайдер) с нормализацией писем в тикеты.
- **Ф. Реалтайм и фон** — WebSocket-шлюз на NestJS Gateways (socket.io/ws); приватная активность не уходит на клиентские соединения. SSE для стриминга. Воркеры BullMQ + расписания (cron) для регламентных задач.
- **Г. Интерфейс** — применяем токены (light/dark переключением класса на :root); состояние на Pinia; типизированный слой API-клиента. Экраны на shadcn-vue + headless-библиотеки: TanStack Table, виртуализация длинных списков, Tiptap для богатого ввода.
- **Н. Аналитика** — встроенные дашборды на ECharts (vue-echarts), темизированные токенами; агрегаты считаются на бэкенде (SQL / Prisma). Позже — self-hosted Metabase (только чтение, изолированный).
- **И. Готовность к прод (гейт)** — закрыть security-блокеры; egress allow-list на фаерволе; шифрованные бэкапы Postgres (по расписанию, off-host); скрабинг PII в логах; политика retention. До прохождения гейта реальные данные не заводятся.

- **J. Миграция из Zendesk** — выгрузка через Zendesk REST API (пагинация): тикеты, пользователи, организации, поля, макросы, триггеры, представления; маппинг в нашу схему; импорт и сверка на staging.
- **K. AI-фаза (после ядра)** — отдельный сервис: классификация тикетов (категория/подкатегория); подсказки ответов из базы знаний со стримингом (SSE); контроль расходов на модель.

7. После MVP — расширение

MVP — прочная основа. Архитектура модульная, API-ориентированная и white-label, поэтому легко наращивается:

- Новые каналы — WhatsApp, Telegram и другие мессенджеры/соцсети сверх стартового набора.
- Глубже Zendesk — любые дополнительные фичи/настройки Zendesk по мере надобности.
- Самообслуживание и оценка — база знаний / help-center и опросы удовлетворённости (CSAT).
- AI-функции — авто-классификация тикетов и подсказки ответов из базы знаний.
- Интеграции — связи с другими внутренними системами через API и вебхуки.
- Дополнительные бренды — новые внутренние бренды, у каждого своя тема, на той же платформе.

Итог: MVP — фундамент; фичи и интеграции (из Zendesk или новые) добавляются поверх без переделки.

8. Почему такие решения (обоснования)

Краткое «почему» по ключевым решениям (полные обоснования — в журнале решений проекта):

- **Chatwoot как чертёж, не форк** — берём готовую доменную модель поддержки, чтобы не проектировать с нуля, но не наследуем чужую кодовую базу, лицензию и архитектуру целиком.
- **Бэкенд на NestJS / TypeScript (не Ruby)** — исходный Chatwoot на Ruby, а его некому поддерживать вдолгую; TypeScript — язык команды и единый с фронтендом.
- **Фронтенд остаётся Vue (не React)** — команда знает Vue; переписывать сотни компонентов на React — та же проблема «некому поддерживать» плюс потеря готовой логики.
- **shadcn-vue и дизайн-токены** — бесплатно (MIT), код у нас (нет привязки к вендору), брендинг через CSS-переменные, светлая и тёмная темы.
- **Модульный монолит (не микросервисы)** — микросервисы не ускоряют код, а добавляют сложность и латентность; для MVP и небольшой команды это

преждевременно. Отдельные сервисы выносим точно по метрикам (первый кандидат — AI-сервис).

- **Self-hosted Supabase и Redis** — данные остаются у нас; готовая авторизация надёжнее самописной; NestJS остаётся «мозгом» логики.
- **Prisma для данных, supabase-js для авторизации и файлов** — ORM удобнее для серверной логики; миграции ведёт один инструмент.
- **REST плюс SSE / WebSocket (без GraphQL)** — проще и совпадает с фронтом; стриминг закрывают SSE и WebSocket; GraphQL — лишняя сложность.
- **Моно-аренда с заделом на мульти-бренд** — сейчас одна компания, но модель аккаунтов сохранена как «шов»: мульти-бренд и white-label включаются позже дёшево, без потери изоляции.
- **Своя система ролей (RBAC)** — гранулярные роли в Chatwoot были платными (Enterprise); нам нужен собственный контроль доступа.
- **Веб-приложение (десктоп — позже)** — доступ отовсюду, ничего не устанавливать, мгновенные обновления; десктоп сейчас не даёт выгоды.
- **Support-first, API-канал ключевой** — суть системы — обработка обращений; через API уже работает наш внешний AI-виджет и поступают сообщения из мессенджеров.
- **Self-host, Docker Compose, Caddy** — данные в-хаус, дёшево, автоматический HTTPS; облако и Kubernetes избыточны на старте.
- **GitHub Actions с тест-гейтом** — деплой без ручной заливки; тесты защищают продакшн.
- **Безопасность строится с нуля** — система хранит персональные данные; внешний аудит переоценён под нас, блокиеры закрываются до реальных данных.
- **Бэкапы, egress-allowlist, PII вне логов, retention** — сохранность и комплаенс; данные не утекают наружу.
- **Встроенная аналитика (ECharts), Metabase позже** — брендированные дашборды в интерфейсе; бесплатный Metabase не темизируется под бренд, поэтому это внутренний инструмент на потом.
- **AI-фаза после ядра** — внешний виджет уже закрывает первую линию; встроенный AI зависит от готовой модели данных и решается позднее.
- **Тесты как обязательный гейт** — объективная проверка качества и защита от регрессий.